

FileRerankingBench: A Small Benchmark for File Selection in Code-Editing Agents

Ahmad Jiha

Daniel Chen
Anything, Inc.

Abstract

Code-editing LLM agents need to select a small set of relevant project files before editing. FileRerankingBench is a small public benchmark for that file-selection step. The benchmark contains prompt-level relevance labels, candidate file metadata, a project corpus, and aggregate evaluation numbers. The public artifact is intentionally limited to benchmark data, aggregate results, and this paper.

1 Motivation

Code-editing agents often operate on projects that are too large to place entirely in model context. Before the model edits code, a system must decide which files are likely to matter for the user’s request. Poor file selection can cause the agent to miss the relevant file, edit the wrong file, or waste context on unrelated boilerplate.

FileRerankingBench isolates this pre-edit file-selection problem. Each benchmark case provides a user prompt, a set of candidate files with timestamp metadata, and the expected relevant file paths.

2 Dataset

The current public dataset contains one small hiking-app project and five query fixtures. The cases cover:

- direct lexical matches between the prompt and target page,
- stale or misleading recency metadata,
- paraphrased references to UI text,
- and a prompt whose requested page does not exist in the candidate set.

The fixture file is `data/file-reranking-evals.json`. The project corpus is `data/projects/hiking-app-sm`. Candidate paths in the fixture are app-root paths, for example `/apps/web/pages/index/+Page.jsx`; the zip stores files below `createxyz-project/`.

3 Metrics

The benchmark reports Recall@ k and Precision@ k for $k \in \{1, 3, 5, 10\}$. Let T_k be the top- k returned files and let R be the set of expected relevant files.

$$\text{Recall@}k = \begin{cases} 1 & \text{if } |R| = 0, \\ \frac{|T_k \cap R|}{|R|} & \text{otherwise.} \end{cases}$$

$$\text{Precision@}k = \frac{|T_k \cap R|}{|T_k|}$$

If fewer than k candidate files are available, T_k contains only the available candidates.

4 Published Results

Table 1 reports aggregate reference numbers over the current public fixture set. These numbers are intended to anchor the benchmark and make future comparisons explicit without expanding the public artifact beyond data, results, and this paper.

System	Recall@1	Precision@1	Recall@3	Precision@3
Original candidate order	0.6000	0.4000	1.0000	0.2667
Recency baseline	0.4000	0.2000	1.0000	0.2667
Lexical baseline	0.6000	0.4000	1.0000	0.2667
Rank-all oracle upper bound	1.0000	0.8000	1.0000	0.2667

Table 1: Macro-averaged results over five public benchmark cases. Recall@5, Precision@5, Recall@10, and Precision@10 match the Recall@3 and Precision@3 columns in this corpus because each case has three candidate files.

5 Release Contents

The public repository at <https://github.com/Create-Inc/file-reranking-bench> contains:

- the benchmark fixture JSON,
- the project corpus zip,
- aggregate result tables,
- and this paper.

Production systems are out of scope for this artifact.

6 Limitations

The current corpus is intentionally small: five queries over one project. It is best understood as a regression benchmark and publication artifact, not a broad statistical measurement of file-selection quality. Future versions should add more projects, more user prompts, multi-file relevance labels, and richer negative cases.

7 Conclusion

FileRerankingBench provides a compact public artifact for evaluating the file-selection step in code-editing agents. By publishing the data and aggregate results, the benchmark can support external comparison while keeping production systems out of scope.